Miércoles 11/11: preparar la demo del jueves

Lista de tareas para el laboratorio, medidas para la memoria, vídeos y comandos de prueba. Las casillas son para marcar; los nombres de cada tarea son una propuesta según el reparto del Bloque 3 (Javier: demo y GPU; Alejandro: seguimiento y rival móvil; Mónica: circuito; Pedro José: conexión; Daniel: montaje).

Objetivo del miércoles	Para darlo por hecho
Circuito montado, medido y marcado en el suelo	Todas las marcas a lápiz, la tabla de medidas rellena y una foto cenital
Demo completa sin fallos con el robot real	Tres ejecuciones seguidas que atacan a los cuatro objetivos y terminan el baile
Decidir CPU o GPU para el jueves	demo_gpu.py probado en el ordenador que se usará; si no, demo.py
Datos reales para la memoria y la presentación	Tabla de resultados rellena, evidencias copiadas y vídeos grabados
Mando probado con el robot real	Pruebas cortas del mando hechas y retardo del vídeo medido

1. Antes de ir al laboratorio (martes por la noche o miércoles temprano)

- ☐ Cargar las baterías: nuestro robot, los robots rivales y el mando de PS4. Llevar los cargadores. · todos
- ☐ Imprimir el robot camuflado en A3 y papel mate: *docs/informes/figuras/senuelo/detective_imprimir.jpg* (unos 35 cm de alto, como la caja de un RoboMaster). Llevar algo rígido para que no se doble (cartón o carpeta). · Javier
- ☐ Material: cinta métrica de 5 m o más, lápiz, cinta de carrocero, rotulador, una plataforma de 20 a 40 cm para el rival 3, regleta, cable USB del robot y un móvil con trípode o apoyo para grabar. · Daniel
- ☐ Portátil: *git pull* de la rama, comprobar que está el modelo (SHA-256 abajo) y que pasan las pruebas. · Javier
- ☐ Probar en casa la demo en simulación con las dos versiones: `python demo.py --simulacion` y `python demo_gpu.py --simulacion`. La segunda debe decir «Detector en la GPU NVIDIA GeForce RTX 5070…». · Javier
- ☐ Portátil del rival 2: entorno de Python 3.8 con el SDK y *rival/rival_movil.py --sin-robot* probado. · Alejandro
- ☐ Copia de seguridad: el modelo *model_demo.pt* en un USB, por si hay que usar un ordenador del laboratorio. · Javier

2. Montar el circuito, medirlo y marcarlo a lápiz

El plano de *circuito.py* tiene el origen en la salida de nuestro robot, con x hacia delante (derecha del boceto) e y hacia abajo. Solo P1 está medido; el resto son supuestos. El objetivo es que el jueves el circuito se pueda volver a montar igual en 10 minutos usando las marcas.

- ☐ Marcar la **salida**: una cruz en el centro del robot y una flecha hacia donde mira (RUMBO_INICIAL = 0). Marcar también las cuatro ruedas, para colocarlo siempre igual. · Mónica, Daniel
- ☐ Medir y marcar **P1** a 2,85 m en línea recta (ya comprobado el 05/11: unos 6 s). · Mónica, Daniel
- ☐ Marcar al **rival 1** 1,2 m a la derecha de P1 y al **robot camuflado** 1,2 m a la izquierda, a la altura de la cámara si se puede (el A3 de pie, sin brillos de los fluorescentes). · Mónica, Daniel
- ☐ Marcar el final de cada movimiento de **R1**: 1,9 m adelante, 1 m a la izquierda, 1,25 m atrás y 1 m a la derecha. · Mónica, Daniel
- ☐ Marcar el **rival 2**: empieza 1,2 m a la izquierda del retroceso de R1 y recorre 0,75 m en el mismo sentido (marcar inicio y final de su recorrido). · Alejandro
- ☐ Marcar **R2**: 2,5 m adelante, 0,7 m a la izquierda y 2 m adelante despacio. Colocar al **rival 3** detrás, en la plataforma, y medir su altura exacta. · Mónica, Daniel
- ☐ Junto a cada marca, escribir el nombre (P1, R1-a, rival 1…) en cinta de carrocero, no en el suelo. · Daniel
- ☐ Comprobar que hay al menos 0,5 m libres alrededor de todo el recorrido y que nadie se pone en la trayectoria. · todos
- ☐ Hacer una **foto cenital** del circuito montado (de pie en una silla o desde lo alto) para la memoria y la presentación, y otra con los robots colocados. · Pedro José

- [ ] Si el circuito real no coincide con el plano, cambiar *RIVALES* y *PLANO* en *circuito.py* y revisar con `python recorrido.py` antes de mover el robot. · Javier

Las medidas se apuntan en la tabla del apartado 9 (sirve tal cual para la Tabla B3.1 de la memoria).

3. Elegir el ordenador: portátil (RTX 5070) o laboratorio (RTX 3060)

- [ ] **Opción por defecto: el portátil de Javier.** Ya tiene los dos entornos y *demo_gpu.py* probado. · Javier
- [ ] En un ordenador del laboratorio (Linux), ver si el PyTorch de pixi ya usa la GPU: `…/.pixi/venv/bin/python -c "import torch; print(torch.cuda.is_available())"`. Si da True, *demo.py* ya va en la 3060. · Pedro José
- [ ] Si da False y se puede instalar: crear *.venv-gpu* (comandos del apéndice) y medir con `detector_gpu.py --medir 10`. Si no se puede instalar en 15 minutos, quedarse con el portátil. · Javier
- [ ] Anotar los ms por imagen de cada opción (tabla del apartado 9). Es un dato para la memoria (apartado 13). · Javier

4. Conexión y calibración en P1, sin mover ni disparar

- [ ] Conectar por wifi directo (portátil en la red RMEP-… del robot) o por networking (ordenadores del laboratorio, con *connect.py* rellenado solo en local). Debe salir «Conexion: wifi» o «Conexion: networking». · Pedro José
- [ ] Robot en la salida, llevado a mano hasta P1. `python demo_gpu.py --sin-mover --sin-disparar --paradas P1 --ver`. Comprobar que ve al rival 1 a la derecha y al robot camuflado a la izquierda. · Javier
- [ ] Si la torreta gira hacia el lado contrario, poner *SIGNO_YAW = -1* en *circuito.py*. · Javier
- [ ] Revisar las confianzas en *evidencias/*. Si el camuflado se queda por debajo de 0,95, acercarlo, quitar brillos o probar con *--umbral 0.9* y anotar el motivo (pendiente del apartado 16.1 de la memoria). · Javier
- [ ] Comprobar el aviso de voz, el sonido de la SCAR y los LED (ámbar buscando, rojo disparando, verde eliminado). · Daniel

5. Puntería y disparo con el robot quieto

- [ ] `python demo_gpu.py --sin-mover --paradas P1`: debe centrar al rival 1 y disparar la ráfaga de 20. · Javier
- [ ] Contar los **impactos que registra el rival 1** (el enunciado pide 12) y el número de correcciones de la torreta antes de disparar, con el error final (está en *evidencias/*). Es el pendiente del apartado 15.1. · Alejandro
- [ ] Repetir con *demo.py* (CPU) para ver si el fallo del 04-05/11 («detecta pero no dispara») está resuelto en las dos versiones. · Javier

6. Recorrido completo, primero sin disparar

- [ ] `python demo_gpu.py --sin-disparar` con alguien junto al robot listo para Ctrl+C. · Javier, Daniel
- [ ] Medir con la cinta dónde se para el robot en cada marca y cuánto se desvía (cm). Cronometrar cada tramo con el móvil. · Mónica, Daniel
- [ ] Ataque en marcha de R1 con el **rival 2 en movimiento**: en el portátil del rival, `python rival/rival_movil.py`; en el nuestro, `python demo_gpu.py --aviso-rival IP`. Comprobar que el rival arranca solo al empezar el retroceso. · Alejandro
- [ ] R2 con el **rival 3 en alto**: comprobar que la torreta lo busca inclinada y que lo centra en vertical. · Javier
- [ ] Si un tramo falla, probarlo suelto con *--paradas* antes de repetir todo el recorrido. · Javier

7. Activar las mejoras de una en una

Cada mejora se activa en *circuito.py*, se prueba con *--sin-mover --sin-disparar* y después en la demo completa. Si una falla, se vuelve a poner a *False* y la demo sigue funcionando sin ella.

- [ ] LEER_ANGULO_GIMBAL = True: al arrancar debe salir «Torreta: lectura del ángulo comprobada». · Javier
- [ ] SEGUIMIENTO = True: alguien mueve un robot delante; la torreta lo sigue y dispara la ráfaga de 20. Con GPU la ráfaga dura unos 6 s; con CPU, unos 23 s. · Alejandro

- [ ] BARRIDO_CONTINUO = True: comprobar en *evidencias/* que detecta con la torreta girando; si no, bajar VEL_BARRIDO. · Alejandro
- [ ] MEMORIA = True, después de calibrar ALTURA_ROBOT (robot a 1,5 m y mirar distancia_m). · Alejandro
- [ ] VIGILANCIA = True, la última. · Alejandro
- [ ] Decidir qué mejoras quedan activadas el jueves y apuntarlo (también para la memoria). · todos

8. Demo completa y resultados

- [ ] Tres ejecuciones completas con `python demo_gpu.py --aviso-rival IP` (o *demo.py* si se eligió CPU). · Javier
- [ ] Rellenar la tabla de resultados del apartado 9 en cada ejecución: impactos por rival, falsos positivos, robots no vistos y tiempo total. · Mónica
- [ ] Guardar las carpetas de *evidencias/* de las buenas en un sitio compartido con un nombre claro (no se suben a git). · Pedro José
- [ ] Ensayar la colocación rápida: desmontar los rivales, volver a montarlos con las marcas y cronometrarlo. · todos

9. Tablas para rellenar

Medidas del circuito (Tabla B3.1 de la memoria)

Elemento	Previsto	Medido	Tiempo	Desvío al parar / notas
Salida → P1 (recta)	2,85 m a 0,5 m/s			
Rival 1 (derecha de P1)	1,2 m		—	
Robot camuflado (izquierda de P1)	1,2 m		—	
R1 · adelante	1,9 m a 0,4 m/s			
R1 · izquierda	1 m a 0,5 m/s			
R1 · atrás atacando (rival 2)	1,25 m a 0,1 m/s			
R1 · derecha	1 m a 0,5 m/s			
Rival 2 · separación y recorrido	1,2 m · 0,75 m a 0,05 m/s			
R2 · adelante	2,5 m a 0,5 m/s			
R2 · izquierda	0,7 m a 0,5 m/s			
R2 · adelante atacando (rival 3)	2 m a 0,1 m/s			
Rival 3 · posición y altura	(5,35; −0,7) m · 20-40 cm		—	

Tiempo por imagen

Ordenador	Versión	ms por imagen	GPU que indica	Notas
Portátil de Javier	demo.py (CPU)		—	
Portátil de Javier	demo_gpu.py		RTX 5070 Laptop	
Ordenador del laboratorio	demo.py / demo_gpu.py		RTX 3060	

Resultados de cada ejecución (apartado 18 de la memoria)

#	Versión	Rival 1 (impactos)	Camuflado	Rival 2	Rival 3	Falsos positivos	No vistos	Tiempo	Notas
1									
2									
3									
4									

10. Mando: pruebas con el robot real

- [ ] Arrancar con `mando\arrancar.cmd --sin-mover` y abrir *http://127.0.0.1:8765* → «Activar controles». · Javier
- [ ] Comprobar los cuatro emotes y cortar uno con Esc y otro con SHARE. · Daniel
- [ ] Mirada y disparo de infrarrojos a un rival con el robot sujeto. · Daniel
- [ ] Robot sobre una caja: sentidos de las ruedas, y que se para al soltar el mando o desconectarlo (0,3 s). · Pedro José
- [ ] En el suelo: cada baile, deriva y sincronía con la música. · Daniel
- [ ] **Retardo del vídeo**: poner un cronómetro del móvil delante de la cámara del robot y hacer con otro móvil una foto que salga a la vez el cronómetro y la pantalla del portátil. La diferencia es el retardo. Repetir 3 veces, por wifi directo y por la red del laboratorio. Es el pendiente del apartado 17 de la memoria. · Pedro José

11. Vídeos y fotos

Grabar en horizontal, 1080p, con el móvil apoyado o en trípode y el sonido activado. Nombrar cada archivo con la hora y lo que se ve. Para la presentación bastan clips cortos (de 10 a 40 s).

- [ ] **Demo completa** en plano fijo desde una esquina alta, donde se vea todo el circuito. · Pedro José
- [ ] **Disparo de cerca**: la torreta centrando al rival 1, la ráfaga con el sonido y los LED del rival. · Mónica
- [ ] **Robot camuflado**: el giro a la izquierda en P1 y la ráfaga contra la hoja impresa. · Mónica
- [ ] **Ataque en marcha**: R1 con el rival 2 moviéndose en paralelo (es el momento más vistoso). · Mónica
- [ ] **Pantalla del portátil** grabada (Win+Alt+R o OBS) con la ventana de *--ver* y las detecciones. · Javier
- [ ] **Mando**: 20 s manejando el robot y un baile. · Daniel
- [ ] Fotos: circuito montado desde arriba, robot en cada parada, la hoja del camuflado colocada y el equipo. · Pedro José
- [ ] Pasar todo esa misma tarde a la carpeta compartida. · todos

12. Después del laboratorio: memoria y presentación

Memoria (cuerpo de 28 páginas de 30: quedan unas 2 para los resultados)

- [ ] 13: tiempo por imagen medido en el ordenador de la demo y si se usa la GPU. · Javier
- [ ] 14, Tabla B3.1: medidas reales de R1 y R2, posiciones de los rivales y foto del circuito montado. · Mónica
- [ ] 15.1: si dispara con la cámara corregida, número de correcciones y error final. · Javier
- [ ] 15.2: ataque en marcha con el rival 2 moviéndose e impactos registrados por cada rival. · Alejandro
- [ ] 16.1: robot camuflado visto desde P1 con la cámara del robot. · Javier
- [ ] 17: mando con el robot real (caja y suelo) y retardo del vídeo. · Pedro José
- [ ] 18: resultados del circuito (tabla de ejecuciones) y qué mejoras quedaron activadas. · Javier
- [ ] 19: revisar las conclusiones con los resultados. · todos
- [ ] Numerar tablas y figuras, actualizar el índice (F9), regenerar el PDF y comprobar que el cuerpo no pasa de 30 páginas. · Javier

Presentación (20 diapositivas)

- [ ] Diapositiva del circuito: ajustar la animación a las distancias y tiempos medidos. · Javier
- [ ] Ataque en marcha: añadir el clip de R1 con el rival 2 y los impactos medidos. · Alejandro
- [ ] Mando: cambiar la imagen simulada por una real y añadir el retardo medido. · Pedro José
- [ ] Añadir las fotos del circuito montado y, si da tiempo, un clip de la demo completa al final de la fase 3. · Mónica
- [ ] Ensayo completo cronometrado con el reparto de quién habla en cada diapositiva. · todos

13. Jueves por la mañana, antes de la demo

- ☐ Baterías cargadas y robot encendido 10 minutos antes; portátil enchufado y sin actualizaciones pendientes.
- ☐ Montar el circuito con las marcas del miércoles y medir P1 otra vez (2,85 m).
- ☐ Prueba rápida: `python demo_gpu.py --sin-mover --sin-disparar --paradas P1`.
- ☐ Rival 2 con su portátil preparado y la IP apuntada; la hoja del camuflado bien sujeta.
- ☐ Plan B a mano: si la GPU falla, la demo sigue sola en CPU; si demo_gpu.py da problemas, lanzar demo.py.
- ☐ Alguien grabando la demo del jueves también.

Apéndice: comandos para probarlo todo

Desde *ENTREGA 2/WEEK3*. En el portátil (Windows), *python* es *.venv\Scripts\python.exe* (Python 3.8 con el SDK). En los ordenadores del laboratorio (Linux) es */home/PIDS/robomaster/.pixi/venv/bin/python* y las rutas van con «/».

Preparar y comprobar el portátil

`git switch javier-saguar git pull`	Últimos cambios de la rama
`gh release download modelo-demo-v1 --repo javiersaguar/PIDS-Proyecto2 -p model_demo.pt -D models/faster_rcnn_v2_aug_robusto_v1_lr001`	Descargar el modelo si falta
`certutil -hashfile models\faster_rcnn_v2_aug_robusto_v1_lr001\model_demo.pt SHA256`	Debe dar f46a49d7c8cb2ced…5b63141
`python -m unittest discover -s tests`	81 pruebas de la demo
`python -m unittest discover -s mando\tests`	31 pruebas del mando
`python recorrido.py`	Movimientos y barridos calculados del plano
`python demo.py --simulacion`	Demo completa sin robot, en CPU
`python demo_gpu.py --simulacion`	Igual con la GPU; dice qué GPU usa
`.venv-gpu\Scripts\python.exe mando\detector_gpu.py --medir 10`	GPU y ms por imagen

GPU en un ordenador del laboratorio (Linux, RTX 3060)

`/home/PIDS/robomaster/.pixi/venv/bin/python -c "import torch; print(torch.__version__, torch.cuda.is_available())"`	True: demo.py ya usa la 3060
`nvidia-smi`	«CUDA Version» 12.8 o mayor
`uv venv --python 3.11 .venv-gpu`	Entorno del detector
`uv pip install --python .venv-gpu/bin/python torch==2.7.1 torchvision==0.22.1 --index-url https://download.pytorch.org/whl/cu128`	Con un controlador antiguo: …/whl/cu118
`uv pip install --python .venv-gpu/bin/python -r mando/requirements-gpu.txt`	Resto de dependencias
`.venv-gpu/bin/python mando/detector_gpu.py --medir 10`	Debe decir RTX 3060

Conexión con el robot

`(wifi directo) conectar el portátil a la red RMEP-… del robot`	Contraseña en la pegatina; sin connect.py
`python connect.py`	Networking: enseñar el QR al robot (SSID y clave solo en local)
`python demo.py --conexion wifi … / --conexion networking …`	Forzar el modo si el automático falla

Calibración y pruebas por partes

`python demo_gpu.py --sin-mover --sin-disparar --paradas P1 --ver`	Ver detecciones en P1 sin moverse
`python demo_gpu.py --sin-mover --paradas P1`	Puntería y ráfaga de 20 con el robot quieto
`python demo_gpu.py --sin-mover --sin-disparar --paradas P1 --umbral 0.9`	Probar otro umbral (anotar el motivo)
`python demo_gpu.py --sin-disparar`	Recorrido completo sin disparar
`python demo_gpu.py --sin-disparar --paradas R1`	Solo hasta R1
`python demo_gpu.py --solo-cpu --sin-mover --paradas P1`	Igual que demo.py, para comparar

Rival 2 en movimiento

`python rival/rival_movil.py --sin-robot`	Portátil del rival: probar sin robot
`python rival/rival_movil.py`	Portátil del rival: esperar el aviso de la demo
`python rival/rival_movil.py --modo enter`	Arrancarlo a mano con Enter si falla la red
`python demo_gpu.py --aviso-rival IP_DEL_PORTATIL_RIVAL`	Nuestro portátil: avisa al empezar el retroceso de R1

Demo completa

`python demo_gpu.py --aviso-rival IP`	Versión GPU con la CPU de respaldo
`python demo.py --aviso-rival IP`	Versión original en CPU (plan B)
`evidencias\AAAAMMDD_HHMMSS\resumen.json`	Resumen por tramo y detector usado (clave detector)

Mando y robot camuflado

`mando\arrancar.cmd --sin-mover`	Mando sin mover el chasis; abrir http://127.0.0.1:8765
`mando\arrancar.cmd`	Mando completo
`http://127.0.0.1:8765/prueba_mando.html`	Probar el DualShock en Chrome o Edge
`python probar_senuelo.py ../../docs/informes/figuras/senuelo/detective_imprimir.jpg`	Confianza del detector con la hoja del camuflado

Seguridad: agarrar el robot siempre por la base, nunca por la cabeza; no apuntar a nadie con el blaster; alguien con la mano en Ctrl+C durante todo recorrido nuevo.